

Evidencias - Interpolación de Newton

Datos del estudiante

Estudiante: Andrey Felipe Pinto Uribe

Código: 0192177

Entorno recomendado: PyCharm / Python

Objetivo

El programa implementa la interpolación de Newton usando diferencias divididas.

Archivos de entrega

Código fuente: Metodo_Interpolacion_Newton_0192177_Andrey_Felipe_Pinto_Uribe.py

Documento PDF: Evidencias_Interpolacion_Newton_0192177_Andrey_Felipe_Pinto_Uribe.pdf

Interpolación de Newton

Evidencia del desarrollo del código fuente

```
1  """
2  Interpolación de Newton
3  Estudiante: Andrey Felipe Pinto Uribe
4  Código: 0192177
5
6  Archivo preparado para entrega en Moodle. Contiene la implementación,
7  un ejemplo de ejecución y validaciones básicas de entrada.
8  """
9
10 from typing import List
11
12
13 def diferencias_divididas(x: List[float], y: List[float]) -> List[float]:
14     if len(x) != len(y) or len(x) == 0:
15         raise ValueError("Las listas x e y deben tener la misma longitud y no ser vacías.")
16
17     n = len(x)
18     coeficientes = y[:]
19
20     for j in range(1, n):
21         for i in range(n - 1, j - 1, -1):
22             denominador = x[i] - x[i - j]
23             if denominador == 0:
24                 raise ValueError("Los valores de x no deben repetirse.")
25             coeficientes[i] = (coeficientes[i] - coeficientes[i - j]) / denominador
26
27     return coeficientes
28
29
30 def evaluar_newton(x_nodos: List[float], coeficientes: List[float], x: float) -> float:
31     resultado = coeficientes[-1]
32     for i in range(len(coeficientes) - 2, -1, -1):
33         resultado = resultado * (x - x_nodos[i]) + coeficientes[i]
34     return resultado
35
36
37 def main() -> None:
38     x_nodos = [0.0, 1.0, 2.0]
39     y_nodos = [1.0, 2.0, 5.0]
40     x = 1.5
41
42     coeficientes = diferencias_divididas(x_nodos, y_nodos)
43     y = evaluar_newton(x_nodos, coeficientes, x)
44
45     print("Interpolación de Newton")
46     print(f"Nodos x: {x_nodos}")
47     print(f"Valores y: {y_nodos}")
48     print(f"Coeficientes: {[round(valor, 6) for valor in coeficientes]}")
49     print(f"Valor interpolado en x = {x}: {y:.6f}")
50
51
52 if __name__ == "__main__":
53     main()
```

Evidencia de ejecución y resultados obtenidos

```
python Metodo_Interpolacion_Newton_0192177_Andrey_Felipe_Pinto_Uribe.py
```

```
Interpolación de Newton
Nodos x: [0.0, 1.0, 2.0]
Valores y: [1.0, 2.0, 5.0]
Coeficientes: [1.0, 1.0, 1.0]
Valor interpolado en x = 1.5: 3.250000
```

Validación del funcionamiento

Proceso realizado

Se desarrolló el algoritmo en Python con funciones separadas, validaciones de datos de entrada y un ejemplo ejecutable desde consola.

Prueba realizada

El programa fue ejecutado localmente y la salida incluida en este PDF corresponde al resultado real obtenido al correr el archivo .py entregado.

Conclusión

La ejecución finaliza sin errores y muestra resultados coherentes para el caso de prueba seleccionado, evidenciando el correcto funcionamiento del método implementado.